iesVirgen de la Paloma
Centro Público de F.P.

Theca

Desarrollo de Aplicaciones Multiplataforma

Trabajo fin de ciclo presentado por:	Javier Pérez Báez
Tutor/a:	Isidoro Nevares Martín
Fecha:	01 de febrero de 2026

IES VIRGEN DE LA PALOMA

Departamento de Informática

Informática y Comunicaciones

3. Contexto actual

3.1. Estado del arte

Este apartado se desarrolla con base en el análisis de cuatro aplicaciones representativas del mercado, con un enfoque basado en sus arquitecturas para contrastarlas con la de este proyecto (un *frontend* de escritorio[1] conectado a una API en la nube). Estas cuatro aplicaciones son Calibre, Sotero, Mendeley y EndNote.

Calibre es un gestor de libros electrónicos monolítico y de escritorio. Su núcleo es una aplicación que gestiona directamente una biblioteca local de archivos, con funciones de conversión, organización y sincronización con lectores físicos.
Sus principales fortalezas son su especialización extrema en un dominio (*eBooks*), una gestión sofisticada de metadatos y formatos, amén de que ya cuenta con una gran comunidad de usuarios. Además, es de código abierto y gratuito.
Sin embargo, su arquitectura es monolítica y cerrada. Esto es, toda la lógica, la interfaz y los datos residen en el escritorio del usuario. No está diseñada para una separación entre el cliente y el servidor que permita un *frontend* ligero o un *backend* escalable en la nube.

Por otro lado, **Zotero** gestiona referencias bibliográficas con un modelo híbrido: combina una aplicación de escritorio, que gestiona una base de datos local, con una sincronización opcional en la nube para copias de seguridad y colaboración.
Destaca porque captura automáticamente los metadatos desde el navegador y porque tiene una fuerte integración con procesadores de texto para citas. Es un modelo *freemium*.

En este caso, aunque sí trabaja en la nube, lo hace sólo para la sincronización, no como parte de la arquitectura de servicios. Es decir, la nube actúa principalmente como espejo de la base local, no como un *backend* de servicios con API central diferentes clientes puedan consumir diferentes clientes de forma nativa. La lógica principal, al fin y al cabo, sigue atada al cliente de escritorio.

[1] Modificar si, al final, se desarrollan más clientes.

Mendeley es un gestor de referencias orientado a la nube, como una suerte de red social académica. Aunque tiene cliente de escritorio, prioriza la nube como repositorio principal, fomentando el descubrimiento y la colaboración.
Presenta, como ventajas, un enfoque colaborativo integrado, recomendaciones basadas en la biblioteca y que, al fin y al cabo, está respaldado por una gran editorial como es Elsevier.
No obstante, su principal desventaja es precisamente su dependencia del proveedor, pues, al ser un servicio cerrado, el usuario depende totalmente de la infraestructura, de las reglas y de la continuidad del proveedor.

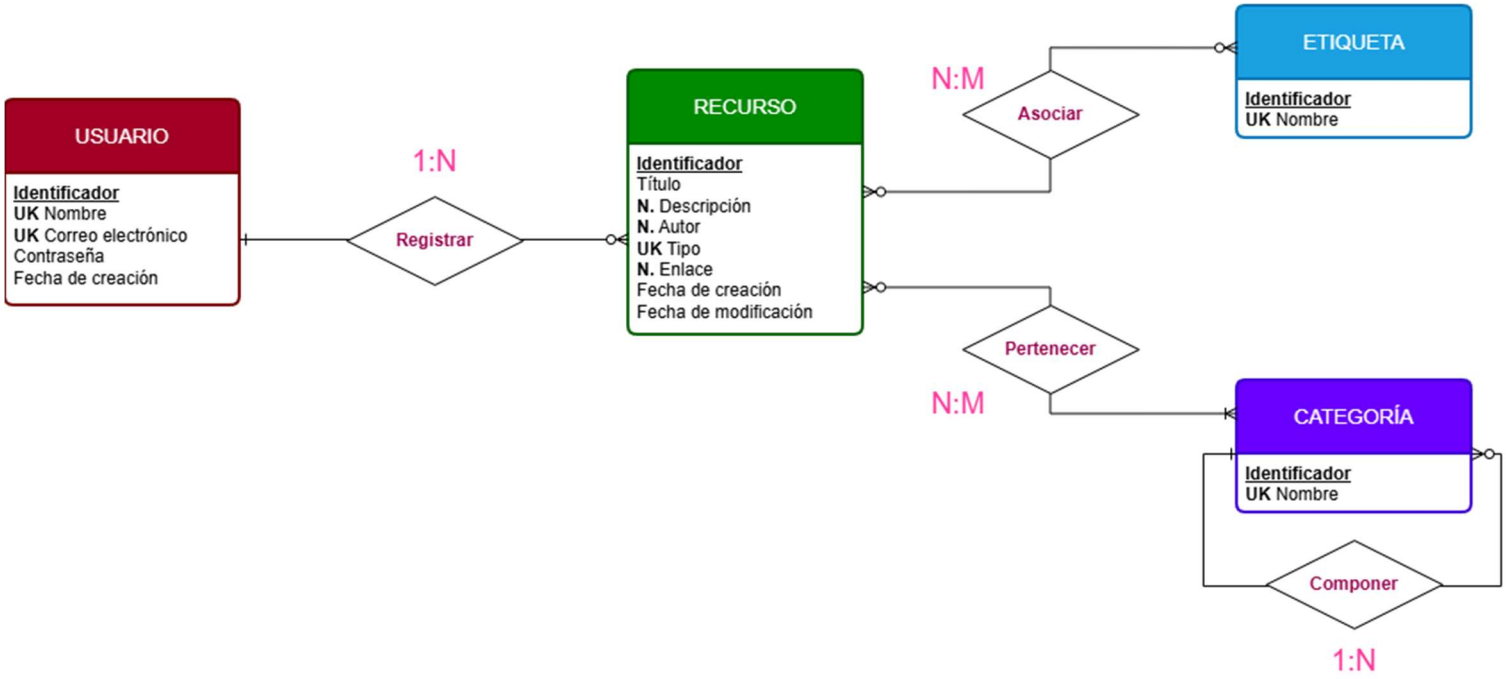
Finalmente, el estándar tradicional comercial en gestión de referencias es **EndNote**, que ofrece un cliente de escritorio robusto y una versión web (EndNote Basic), pero sin una integración profunda entre ambos como un único servicio API.
Se trata de una aplicación con funcionalidades muy completas y pulidas. Cuenta con una amplia adopción en entornos académicos establecidos. Además, presenta un soporte profesional.
Evidentemente, es un modelo propietario y de elevado coste; es un *software* comercial de pago. Su arquitectura está más segmentada (escritorio frente a web) que unificada en un *backend* moderno, lo que la hace menos flexible para desarrollos personalizados o expansiones.

Así pues, el análisis presentado en las líneas anteriores revela que, aunque existen soluciones maduras, ninguna adopta la arquitectura específica que en este proyecto se propone, a saber: un *backend* como servicio en la nube (Spring Boot en AWS) desacoplado de un cliente de escritorio nativo (JavaFX) que se comunique exclusivamente a través de la API.
Esta arquitectura es una ventana de oportunidad porque separa claramente la lógica de negocio de la presentación —el *backend* en AWS es un servicio independiente, escalable y accesible desde cualquier cliente futuro—, combina el control local con la potencia en la nube y, em definitiva, demuestra competencias modernas y profesionales.

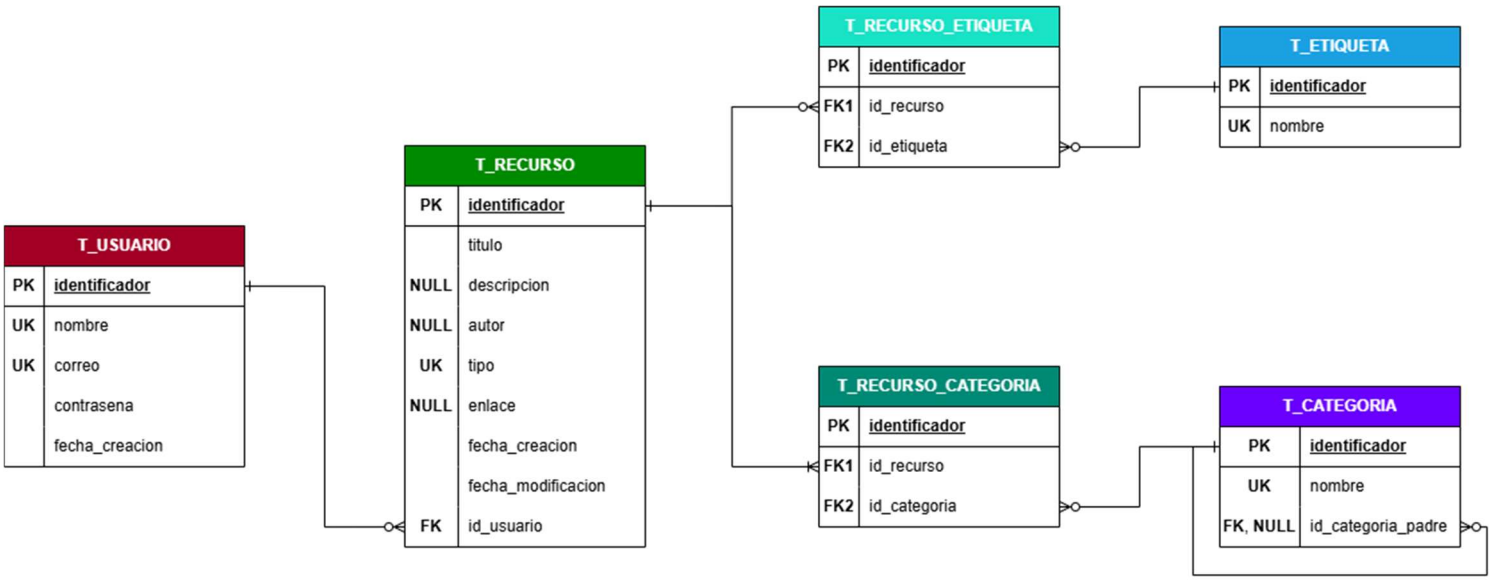
3.2. Conceptos clave

Para investigar o documentar el proyecto, los términos clave que tienen que ver con el mismo son la **arquitectura cliente-servidor/3-Tier**, que es el patrón fundamental del sistema; el **API REST** (concretamente, **Spring Boot**), que se constituye como la capa de comunicación entre el cliente y el servidor; la **computación en la nube** (**AWS**), que no es sino la plataforma donde se aloja y se ejecutan el *backend* y los servicios; **JavaFX**, que es la tecnología usada para el cliente de escritorio; la **gestión de colecciones digitales/metadatos** (el dominio de la aplicación); y los **contenedores** (**Docker**) o los **Servicios AWS** (**EC2**, **Elastic Beanstalk** o **RDS**), que son las tecnologías clave para el empaquetado y el despliegue en la nube.

6.3. Diagrama entidad/relación



6.4. Diagrama Relacional



12. Bibliografía/Webgrafía

- Calibre. (s.f.). *Calibre - Gestión de libros electrónicos*. Recuperado 2 de febrero de 2026, de https://calibre-ebook.com/es
- Corporation for Digital Scholarship. (s.f.). *Zotero | Your personal research assistant*. Recuperado 1 de febrero de 2026, de https://www.zotero.org
- Seatable. (2024, 17 de mayo). *Programa de gestión de la literatura para una mejor visión de conjunto*. Recuperado 1 de febrero de 2026, de https://seatable.com/es/gestion-bibliografica
- Wikipedia contributors. (s.f.). *Comparison of reference management software*. En Wikipedia, The Free Encyclopedia. Recuperado 1 de febrero de 2026, de https://en.wikipedia.org/wiki/Comparison_of_reference_management_software
- Fordham University Libraries. (s.f.). *Bibliographic Management Tools: Which Tool is Best?* Recuperado 1 de febrero de 2026, de https://fordham.libguides.com/BibliographicManagement